

ADR-006: how a fork build updates itself

Status: proposed, 2026-10-31. Nothing is built. Written after Jay asked for "an auto updater for our code" and chose, against the recommendation below, to put it **in the game** rather than beside it. That choice is recorded, and so is what it costs.

What prompted it

The fork is built by hand on ENG-1 and copied to a machine. On 2026-10-31 that machine was `MSI`, and the copy was a 159 MiB zip over SMB. That does not scale past one person who is willing to be told when a build exists.

Measured first, because the obvious plan does not work

A build made by `BuildPlayerHeadless` cannot tell you which build it is.

`GlobalVariables.LoadVersion()` reads `Resources/version.txt` only under `YARG_TEST_BUILD` or `YARG_NIGHTLY_BUILD`. Our headless build sets **no scripting defines** — `BuildOptions.Development` is a build *option*, not a define — so `CurrentVersion` stays the hardcoded `"v0.15"`. An updater that asks the game "what are you?" gets a constant.

Two further consequences of the same fact, both measured on the MSI build:

`PersistentDataPath`	`…/FatalException/YARG-FE/**release**/` — *not* `dev/`, which needs `YARG_TEST_BUILD`
`Assets/Resources/version.txt`	`dev b4180 (679f00d3)` — written on every build by `BuildGitCommitVersion`
that string inside `build/msi/YARG_Data/resources.assets`	**present**

So the identity *ships*; only upstream's accessor refuses to read it. **The updater will read `Resources.Load<TextAsset>("version")` directly** and leave `LoadVersion()` alone. That is both the smaller diff and the one that does not move the data folder out from under an existing install — adding `YARG_TEST_BUILD` to get a readable version would silently relocate settings, song cache and characters from `release/` to `dev/`.

The format is `<branch> b<count> (<shorthash>)`, and `<count>` is `git rev-list --count HEAD`: **monotonic on a branch, and therefore the only field that may be used for ordering.** Comparing hashes tells you *different*, never *newer*.

The recommendation that was not taken

An updater **beside** the game rather than inside it: no Unity code at all, nothing to upstream, and no risk of a half-written client replacing its own files. Jay chose in-game. The reasons against are not fatal, and they are:

1. It is the largest non-upstreamable thing the fork would carry. `RemoteQueue` is defensible to upstream as a feature; "this fork updates itself from Jay's server" is not.
2. **A running process on Windows cannot replace its own files.** In-game therefore still needs an external helper. "In the game" buys the *check and the UI*, not the swap.

So the design below is in-game for everything except the one step that cannot be.

Design

Hosting. A static manifest and zip on vault2, served by the existing web stack — *not* a new endpoint in `yarg-song-server`. The server would need a Go change and a release to gain a feature that nginx already has, and the song server's job is songs.

```
{
  "branch": "dev",
  "build": 4181,
  "commit": "a1b2c3d",
  "url": "http://192.168.2.7:8099/updates/YARG-FE-b4181.zip",
  "sha256": "<64 hex>",
  "size": 166122027,
  "notes": "one line, shown to the player"
}
```

In the game — new code only, in `Assets/Script/Integration/SelfUpdate/`, following `RemoteQueue`'s precedent of owning no scene object and touching nothing existing except one settings row:

1. Read own identity from `Resources/version.txt`. Unparseable → feature disables itself silently.
2. `GET` the manifest on a timeout. **Never block the main menu**; a dead server must cost nothing.
3. Newer iff `manifest.branch == mine && manifest.build > mine`. A *different* branch is never an update.
4. Show it in Settings. Download only on an explicit click — never automatically.
5. Download to `PersistentDataPath/update-staging/`, then **verify sha256 before anything is unpacked**. Mismatch → delete and report. Unpack to a staging folder.
6. Write `update-pending.json` naming the staging folder and the install folder.

The helper — `yarg-update.exe` (or `.cmd`), shipped beside `YARG.exe`:

1. Game spawns it with its own PID and exits.
2. Helper waits for that PID, renames the install folder aside, moves staging into place, relaunches `YARG.exe`, then deletes the old folder. **Any failure restores the rename.**

What this costs, stated plainly

- **An update is arbitrary code execution, over plain HTTP.** A sha256 in the manifest proves the zip matches the manifest; it proves nothing if the manifest itself is spoofed. Anyone who can answer for `192.168.2.7` can run code on every machine running the fork. The honest fixes are HTTPS with a pinned cert, or signing the manifest with a key whose public half is compiled into the client. **Neither is in this design yet, and the feature should not leave the LAN without one.** The song server already accepts plain HTTP on the LAN, but that risk is "someone serves you a bad song", not "someone replaces your executable".
- **603 MB per update**, whole-build replacement. No deltas. Over wifi that is minutes.
- Release binaries are **not signed** (`SYNC-CLIENT.md`), so Windows SmartScreen will have opinions about a freshly swapped exe.
- The fork cannot update a build it did not make: the manifest's `branch` must match, which means a machine on `dev` never silently receives something from another branch.

Open, and deliberately not decided here

- Whether the helper is a tiny Go binary (the project already ships Go, and `yarg-sync` is precedent) or a `.cmd`. Go is signable later and testable; `.cmd` is zero new build steps.
- Whether ENG-1 publishing a build is manual or a script. Until it is a script, "auto updater" means "automatic for the player, manual for Jay".
- Whether to fix `LoadVersion()` upstream-style instead, which is a one-line change but moves every existing install's data folder.